

Disaster_Image_Classification_End_to_End

January 9, 2026

```
[120]: from tensorflow import keras
from tensorflow.keras import layers
import datasets
```

```
[123]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
[121]: #pip install torch torchvision transformers datasets scikit-learn
```

1 Dataset Loading

The LADI v2 dataset is downloaded from the Hugging Face Hub and cached locally. If the dataset is already present in the cache, it is loaded directly without re-downloading. The total dataset size is approximately 54 GB**s).g.

```
[122]: %%capture
from datasets import load_dataset
ds = load_dataset("MITLL/LADI-v2-dataset")

'(ProtocolError('Connection aborted.', RemoteDisconnected('Remote end closed
connection without response'))), '(Request ID:
dce94bbd-50a4-455a-9bcf-38395b08b91b))' thrown while requesting HEAD
https://huggingface.co/datasets/MITLL/LADI-v2-dataset/resolve/main/README.md
Retrying in 1s [Retry 1/5].
```

```
[9]: ds
```

```
[9]: DatasetDict({
  train: Dataset({
    features: ['image', 'bridges_any', 'buildings_any',
'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any',
'flooding_any', 'flooding_structures', 'roads_any', 'roads_damage', 'trees_any',
'trees_damage', 'water_any'],
    num_rows: 8030
  })
  validation: Dataset({
    features: ['image', 'bridges_any', 'buildings_any',
```

```

'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any',
'flooding_any', 'flooding_structures', 'roads_any', 'roads_damage', 'trees_any',
'trees_damage', 'water_any'],
    num_rows: 890
})
test: Dataset({
    features: ['image', 'bridges_any', 'buildings_any',
'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any',
'flooding_any', 'flooding_structures', 'roads_any', 'roads_damage', 'trees_any',
'trees_damage', 'water_any'],
    num_rows: 1049
})
})

```

```
[48]: ds["train"][3]
```

```

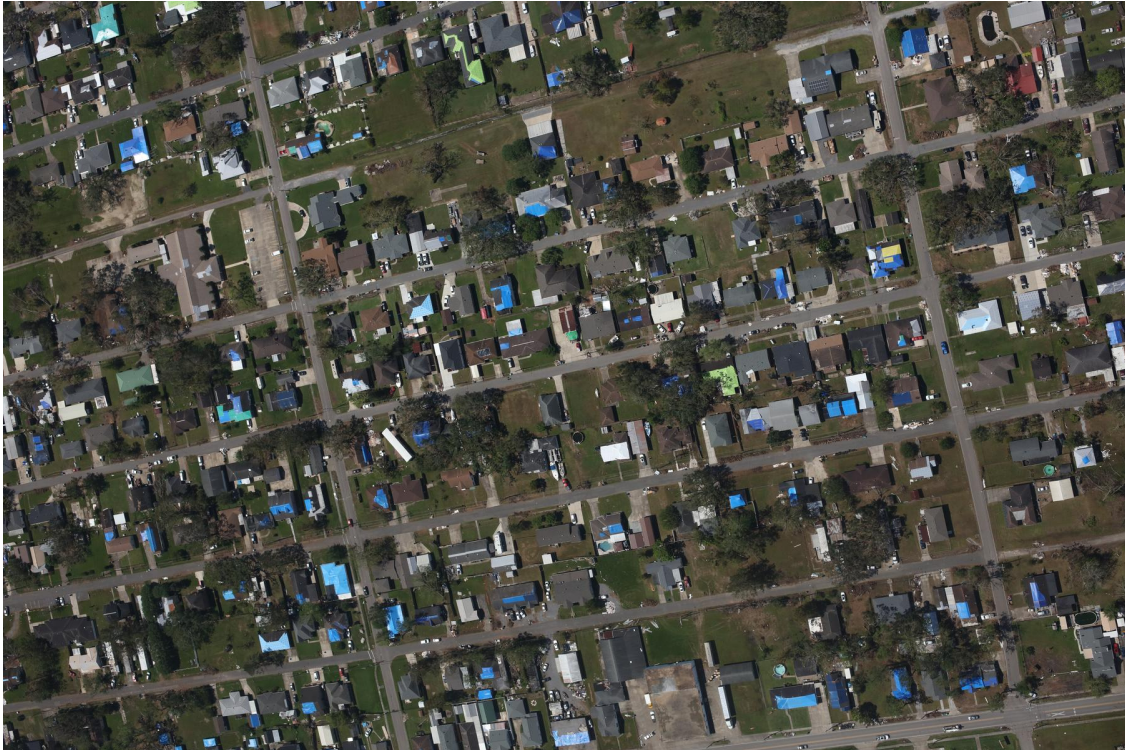
[48]: {'image': <PIL.PngImagePlugin.PngImageFile image mode=RGB size=1800x1200>,
'bridges_any': False,
'buildings_any': True,
'buildings_affected_or_greater': True,
'buildings_minor_or_greater': False,
'debris_any': True,
'flooding_any': False,
'flooding_structures': False,
'roads_any': True,
'roads_damage': False,
'trees_any': True,
'trees_damage': True,
'water_any': False}

```

54.2 GB (58,277,089,280 bytes)

```
[14]: ds["train"][3]["image"]
```

```
[14]:
```



2 1. Model from Scratch

```
[40]: NUM_LABELS = 12
      IMG_SIZE = (180, 180)

      inputs = keras.Input(shape=(180, 180, 3))

      x = layers.Rescaling(1./255)(inputs)

      x = layers.Conv2D(32, 3, activation="relu")(x)
      x = layers.MaxPooling2D(2)(x)

      x = layers.Conv2D(64, 3, activation="relu")(x)
      x = layers.MaxPooling2D(2)(x)

      x = layers.Conv2D(128, 3, activation="relu")(x)
      x = layers.MaxPooling2D(2)(x)

      x = layers.Conv2D(256, 3, activation="relu")(x)
      x = layers.MaxPooling2D(2)(x)

      x = layers.Conv2D(256, 3, activation="relu")(x)
```

```

x = layers.Flatten()(x)

# MULTI-LABEL OUTPUT
outputs = layers.Dense(NUM_LABELS, activation="sigmoid")(x)

model = keras.Model(inputs, outputs)

model.compile(
    optimizer="rmsprop",
    loss="binary_crossentropy",
    metrics=["binary_accuracy"]
)

model.summary()

```

Model: "functional"

Layer (type) ↳Param #	Output Shape	
input_layer (InputLayer) ↳ 0	(None, 180, 180, 3)	↳
rescaling (Rescaling) ↳ 0	(None, 180, 180, 3)	↳
conv2d (Conv2D) ↳896	(None, 178, 178, 32)	↳
max_pooling2d (MaxPooling2D) ↳ 0	(None, 89, 89, 32)	↳
conv2d_1 (Conv2D) ↳18,496	(None, 87, 87, 64)	↳
max_pooling2d_1 (MaxPooling2D) ↳ 0	(None, 43, 43, 64)	↳
conv2d_2 (Conv2D) ↳73,856	(None, 41, 41, 128)	↳
max_pooling2d_2 (MaxPooling2D) ↳ 0	(None, 20, 20, 128)	↳
conv2d_3 (Conv2D) ↳295,168	(None, 18, 18, 256)	↳


```

max_pooling2d_3 (MaxPooling2D)      (None, 9, 9, 256)
↳ 0

conv2d_4 (Conv2D)                  (None, 7, 7, 256)
↳590,080

flatten (Flatten)                  (None, 12544)
↳ 0

dense (Dense)                      (None, 12)
↳150,540

```

Total params: 1,129,036 (4.31 MB)

Trainable params: 1,129,036 (4.31 MB)

Non-trainable params: 0 (0.00 B)

```
[41]: ds["train"], ds["validation"], ds["test"]
```

```

[41]: (Dataset({
    features: ['image', 'bridges_any', 'buildings_any',
'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any',
'flooding_any', 'flooding_structures', 'roads_any', 'roads_damage', 'trees_any',
'trees_damage', 'water_any'],
    num_rows: 8030
}),
Dataset({
    features: ['image', 'bridges_any', 'buildings_any',
'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any',
'flooding_any', 'flooding_structures', 'roads_any', 'roads_damage', 'trees_any',
'trees_damage', 'water_any'],
    num_rows: 890
}),
Dataset({
    features: ['image', 'bridges_any', 'buildings_any',
'buildings_affected_or_greater', 'buildings_minor_or_greater', 'debris_any',
'flooding_any', 'flooding_structures', 'roads_any', 'roads_damage', 'trees_any',
'trees_damage', 'water_any'],
    num_rows: 1049
}))

```

```
[42]: label_names = [
    "bridges_any", "buildings_any",
    "buildings_affected_or_greater", "buildings_minior_or_greater",
    "debris_any", "flooding_any", "flooding_structures",
    "roads_any", "roads_damage",
    "trees_any", "trees_damage", "water_any"
]

[43]: import tensorflow as tf
import numpy as np

def hf_to_tf_dataset(hf_ds, batch_size=32, shuffle=False):
    def gen():
        for sample in hf_ds:
            img = np.array(sample["image"].resize(IMG_SIZE))
            labels = np.array([sample[l] for l in label_names], dtype=np.
↪float32)
            yield img, labels

    output_signature = (
        tf.TensorSpec(shape=(180, 180, 3), dtype=tf.float32),
        tf.TensorSpec(shape=(12,), dtype=tf.float32),
    )

    ds_tf = tf.data.Dataset.from_generator(gen,
↪output_signature=output_signature)

    if shuffle:
        ds_tf = ds_tf.shuffle(1000)

    return ds_tf.batch(batch_size).prefetch(tf.data.AUTOTUNE)

[44]: train_dataset = hf_to_tf_dataset(ds["train"], shuffle=True)
validation_dataset = hf_to_tf_dataset(ds["validation"])
test_dataset = hf_to_tf_dataset(ds["test"])

[47]: callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="convnet_from_scratch.keras",
        save_best_only=True,
        monitor="val_loss"
    )
]

history = model.fit(
    train_dataset,
    epochs=30,
```

```
validation_data=validation_dataset,  
callbacks=callbacks  
)
```

Epoch 1/30

251/Unknown 1095s 4s/step - binary_accuracy: 0.8146 - loss: 0.4839

C:\Users\shahe\Anaconda\Lib\site-packages\keras\src\trainers\epoch_iterator.py:164: UserWarning: Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate at least `steps\_per\_epoch \* epochs` batches. You may need to use the `.repeat()` function when building your dataset.

self._interrupted_warning()

251/251 1211s 4s/step -

binary_accuracy: 0.8266 - loss: 0.4274 - val_binary_accuracy: 0.8411 - val_loss: 0.3853

Epoch 2/30

251/251 1222s 4s/step -

binary_accuracy: 0.8494 - loss: 0.3674 - val_binary_accuracy: 0.8419 - val_loss: 0.3698

Epoch 3/30

251/251 1328s 5s/step -

binary_accuracy: 0.8539 - loss: 0.3526 - val_binary_accuracy: 0.8556 - val_loss: 0.3441

Epoch 4/30

251/251 1363s 5s/step -

binary_accuracy: 0.8572 - loss: 0.3423 - val_binary_accuracy: 0.8528 - val_loss: 0.3484

Epoch 5/30

251/251 1246s 4s/step -

binary_accuracy: 0.8597 - loss: 0.3326 - val_binary_accuracy: 0.8497 - val_loss: 0.3494

Epoch 6/30

251/251 1081s 4s/step -

binary_accuracy: 0.8626 - loss: 0.3253 - val_binary_accuracy: 0.8600 - val_loss: 0.3246

Epoch 7/30

251/251 1117s 4s/step -

binary_accuracy: 0.8663 - loss: 0.3167 - val_binary_accuracy: 0.8605 - val_loss: 0.3240

Epoch 8/30

251/251 1104s 4s/step -

binary_accuracy: 0.8689 - loss: 0.3105 - val_binary_accuracy: 0.8628 - val_loss: 0.3188

Epoch 9/30

251/251 1222s 4s/step -

binary_accuracy: 0.8729 - loss: 0.3015 - val_binary_accuracy: 0.8581 - val_loss: 0.3472

Epoch 10/30
 251/251 6483s 25s/step -
 binary_accuracy: 0.8772 - loss: 0.2921 - val_binary_accuracy: 0.8650 - val_loss: 0.3221

Epoch 11/30
 251/251 1304s 5s/step -
 binary_accuracy: 0.8821 - loss: 0.2816 - val_binary_accuracy: 0.8676 - val_loss: 0.3185

Epoch 12/30
 251/251 1636s 4s/step -
 binary_accuracy: 0.8881 - loss: 0.2700 - val_binary_accuracy: 0.8614 - val_loss: 0.3289

Epoch 13/30
 251/251 1253s 4s/step -
 binary_accuracy: 0.8949 - loss: 0.2546 - val_binary_accuracy: 0.8558 - val_loss: 0.3388

Epoch 14/30
 251/251 1248s 4s/step -
 binary_accuracy: 0.9029 - loss: 0.2381 - val_binary_accuracy: 0.8616 - val_loss: 0.3439

Epoch 15/30
 251/251 1276s 4s/step -
 binary_accuracy: 0.9102 - loss: 0.2178 - val_binary_accuracy: 0.8610 - val_loss: 0.3676

Epoch 16/30
 251/251 1293s 5s/step -
 binary_accuracy: 0.9205 - loss: 0.1966 - val_binary_accuracy: 0.8627 - val_loss: 0.3854

Epoch 17/30
 251/251 3439s 13s/step -
 binary_accuracy: 0.9313 - loss: 0.1687 - val_binary_accuracy: 0.8520 - val_loss: 0.4521

Epoch 18/30
 251/251 952s 3s/step -
 binary_accuracy: 0.9439 - loss: 0.1418 - val_binary_accuracy: 0.8527 - val_loss: 0.5495

Epoch 19/30
 251/251 1110s 4s/step -
 binary_accuracy: 0.9526 - loss: 0.1213 - val_binary_accuracy: 0.8562 - val_loss: 0.6534

Epoch 20/30
 251/251 1557s 4s/step -
 binary_accuracy: 0.9602 - loss: 0.1018 - val_binary_accuracy: 0.8547 - val_loss: 0.6583

Epoch 21/30
 251/251 1164s 4s/step -
 binary_accuracy: 0.9680 - loss: 0.0828 - val_binary_accuracy: 0.8551 - val_loss: 0.7773

```

Epoch 22/30
251/251          1117s 4s/step -
binary_accuracy: 0.9732 - loss: 0.0715 - val_binary_accuracy: 0.8549 - val_loss:
0.8724
Epoch 23/30
251/251          1185s 4s/step -
binary_accuracy: 0.9763 - loss: 0.0644 - val_binary_accuracy: 0.8486 - val_loss:
0.8937
Epoch 24/30
251/251          1171s 4s/step -
binary_accuracy: 0.9794 - loss: 0.0569 - val_binary_accuracy: 0.8422 - val_loss:
1.0129
Epoch 25/30
251/251          1167s 4s/step -
binary_accuracy: 0.9825 - loss: 0.0492 - val_binary_accuracy: 0.8474 - val_loss:
1.0541
Epoch 26/30
251/251          1188s 4s/step -
binary_accuracy: 0.9840 - loss: 0.0461 - val_binary_accuracy: 0.8556 - val_loss:
1.1911
Epoch 27/30
251/251          1149s 4s/step -
binary_accuracy: 0.9860 - loss: 0.0415 - val_binary_accuracy: 0.8508 - val_loss:
1.2340
Epoch 28/30
251/251          1110s 4s/step -
binary_accuracy: 0.9868 - loss: 0.0389 - val_binary_accuracy: 0.8480 - val_loss:
1.3439
Epoch 29/30
251/251          1113s 4s/step -
binary_accuracy: 0.9883 - loss: 0.0341 - val_binary_accuracy: 0.8449 - val_loss:
1.4479
Epoch 30/30
251/251          1148s 4s/step -
binary_accuracy: 0.9894 - loss: 0.0334 - val_binary_accuracy: 0.8515 - val_loss:
1.4680

```

```
[ ]:
```

```

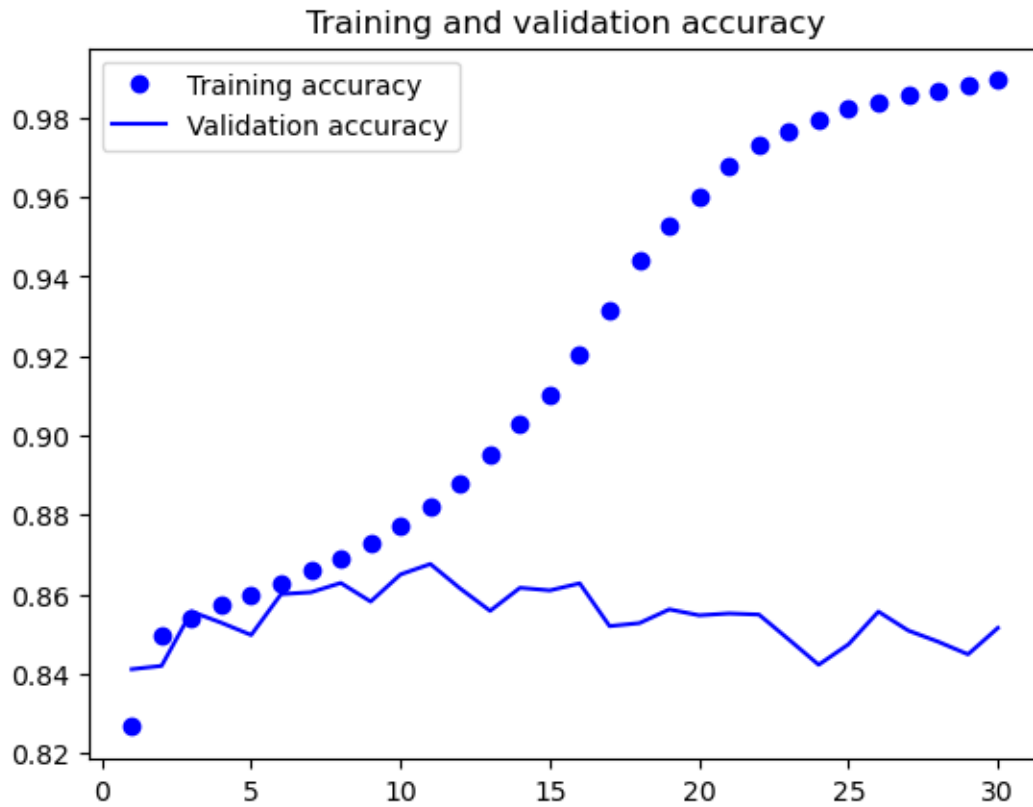
[50]: import matplotlib.pyplot as plt
accuracy = history.history["binary_accuracy"]
val_accuracy = history.history["val_binary_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")

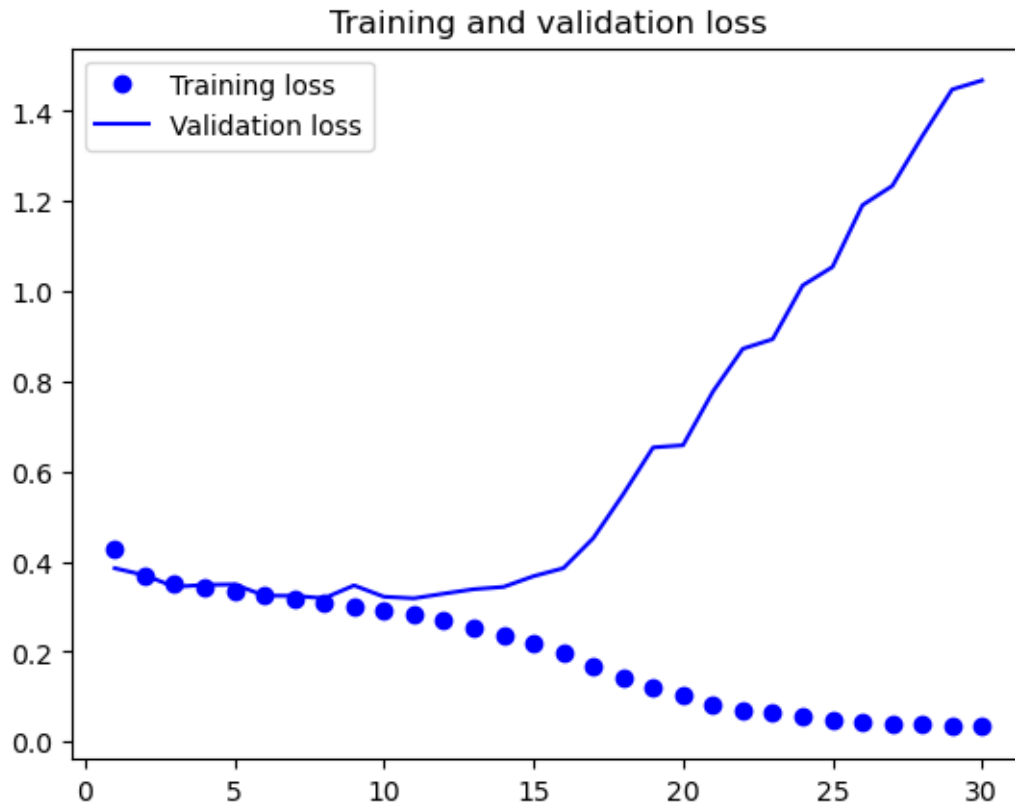
```

```

plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()

```





[]:

```
[51]: test_model = keras.models.load_model("convnet_from_scratch.keras")
test_loss, test_acc = test_model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

```
33/33          138s 4s/step -
binary_accuracy: 0.8724 - loss: 0.2955
Test accuracy: 0.872
```

3 2.Data Augmentation

```
[52]: data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.2),
])
```

```
[58]: plt.figure(figsize=(10, 10))
for images, _ in train_dataset.take(4):
```

```

for i in range(9):
    augmented_images = data_augmentation(images)
    ax = plt.subplot(3, 3, i + 1)
    plt.imshow(augmented_images[0].numpy().astype("uint8"))
    plt.axis("off")

```



```

[53]: NUM_LABELS = 12
      IMG_SIZE = (180, 180)

      inputs = keras.Input(shape=(180, 180, 3))

      x = data_augmentation(inputs)
      x = layers.Rescaling(1./255)(x)

```

```

x = layers.Conv2D(32, 3, activation="relu")(x)
x = layers.MaxPooling2D(2)(x)

x = layers.Conv2D(64, 3, activation="relu")(x)
x = layers.MaxPooling2D(2)(x)

x = layers.Conv2D(128, 3, activation="relu")(x)
x = layers.MaxPooling2D(2)(x)

x = layers.Conv2D(256, 3, activation="relu")(x)
x = layers.MaxPooling2D(2)(x)

x = layers.Conv2D(256, 3, activation="relu")(x)

x = layers.Flatten()(x)
x = layers.Dropout(0.5)(x)

# MULTI-LABEL OUTPUT
outputs = layers.Dense(NUM_LABELS, activation="sigmoid")(x)

model = keras.Model(inputs, outputs)

model.compile(
    optimizer="rmsprop",
    loss="binary_crossentropy",
    metrics=["binary_accuracy"]
)

model.summary()

```

Model: "functional_2"

Layer (type)	Output Shape	
Param #		
input_layer_1 (InputLayer)	(None, 180, 180, 3)	
↪ 0		
sequential (Sequential)	(None, 180, 180, 3)	
↪ 0		
rescaling_1 (Rescaling)	(None, 180, 180, 3)	
↪ 0		

conv2d_5 (Conv2D)	(None, 178, 178, 32)	└
↪896		
max_pooling2d_4 (MaxPooling2D)	(None, 89, 89, 32)	└
↪ 0		
conv2d_6 (Conv2D)	(None, 87, 87, 64)	└
↪18,496		
max_pooling2d_5 (MaxPooling2D)	(None, 43, 43, 64)	└
↪ 0		
conv2d_7 (Conv2D)	(None, 41, 41, 128)	└
↪73,856		
max_pooling2d_6 (MaxPooling2D)	(None, 20, 20, 128)	└
↪ 0		
conv2d_8 (Conv2D)	(None, 18, 18, 256)	└
↪295,168		
max_pooling2d_7 (MaxPooling2D)	(None, 9, 9, 256)	└
↪ 0		
conv2d_9 (Conv2D)	(None, 7, 7, 256)	└
↪590,080		
flatten_1 (Flatten)	(None, 12544)	└
↪ 0		
dropout (Dropout)	(None, 12544)	└
↪ 0		
dense_1 (Dense)	(None, 12)	└
↪150,540		

Total params: 1,129,036 (4.31 MB)

Trainable params: 1,129,036 (4.31 MB)

Non-trainable params: 0 (0.00 B)

```
[54]: callbacks = [
    # Save the best model (lowest val_loss)
    keras.callbacks.ModelCheckpoint(
        filepath="convnet_from_scratch_with_augmentation.keras",
        save_best_only=True,
        monitor="val_loss"
    ),

    # Stop training when validation loss stops improving
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=10,                # stop after 10 epochs with no improvement
        restore_best_weights=True  # load best weights automatically
    )
]
```

```
[55]: history = model.fit(
    train_dataset,
    epochs=100,
    validation_data=validation_dataset,
    callbacks=callbacks
)
```

Epoch 1/100

251/251 1092s 4s/step -

binary_accuracy: 0.8271 - loss: 0.4202 - val_binary_accuracy: 0.8419 - val_loss: 0.3857

Epoch 2/100

251/251 1274s 5s/step -

binary_accuracy: 0.8499 - loss: 0.3668 - val_binary_accuracy: 0.8508 - val_loss: 0.3615

Epoch 3/100

251/251 1308s 5s/step -

binary_accuracy: 0.8523 - loss: 0.3593 - val_binary_accuracy: 0.8525 - val_loss: 0.3498

Epoch 4/100

251/251 1292s 5s/step -

binary_accuracy: 0.8538 - loss: 0.3532 - val_binary_accuracy: 0.8484 - val_loss: 0.3618

Epoch 5/100

251/251 1286s 5s/step -

binary_accuracy: 0.8550 - loss: 0.3477 - val_binary_accuracy: 0.8575 - val_loss: 0.3429

Epoch 6/100

251/251 1307s 5s/step -

binary_accuracy: 0.8570 - loss: 0.3409 - val_binary_accuracy: 0.8537 - val_loss: 0.3437

Epoch 7/100

```

251/251          1297s 5s/step -
binary_accuracy: 0.8589 - loss: 0.3356 - val_binary_accuracy: 0.8530 - val_loss:
0.3568
Epoch 8/100
251/251          1304s 5s/step -
binary_accuracy: 0.8600 - loss: 0.3311 - val_binary_accuracy: 0.8614 - val_loss:
0.3194
Epoch 9/100
251/251          1301s 5s/step -
binary_accuracy: 0.8603 - loss: 0.3285 - val_binary_accuracy: 0.8613 - val_loss:
0.3211
Epoch 10/100
251/251          1309s 5s/step -
binary_accuracy: 0.8632 - loss: 0.3236 - val_binary_accuracy: 0.8535 - val_loss:
0.3380
Epoch 11/100
251/251          1290s 5s/step -
binary_accuracy: 0.8649 - loss: 0.3192 - val_binary_accuracy: 0.8642 - val_loss:
0.3207
Epoch 12/100
251/251          1294s 5s/step -
binary_accuracy: 0.8669 - loss: 0.3146 - val_binary_accuracy: 0.8625 - val_loss:
0.3319
Epoch 13/100
251/251          1293s 5s/step -
binary_accuracy: 0.8688 - loss: 0.3110 - val_binary_accuracy: 0.8620 - val_loss:
0.3348
Epoch 14/100
251/251          1353s 5s/step -
binary_accuracy: 0.8692 - loss: 0.3081 - val_binary_accuracy: 0.8678 - val_loss:
0.3062
Epoch 15/100
251/251          1353s 5s/step -
binary_accuracy: 0.8710 - loss: 0.3048 - val_binary_accuracy: 0.8679 - val_loss:
0.3113
Epoch 16/100
251/251          1301s 5s/step -
binary_accuracy: 0.8720 - loss: 0.3037 - val_binary_accuracy: 0.8680 - val_loss:
0.3092
Epoch 17/100
251/251          1305s 5s/step -
binary_accuracy: 0.8740 - loss: 0.3003 - val_binary_accuracy: 0.8692 - val_loss:
0.3127
Epoch 18/100
251/251          1295s 5s/step -
binary_accuracy: 0.8749 - loss: 0.2962 - val_binary_accuracy: 0.8670 - val_loss:
0.3124
Epoch 19/100

```

251/251 1336s 5s/step -
 binary_accuracy: 0.8758 - loss: 0.2949 - val_binary_accuracy: 0.8738 - val_loss:
 0.2967
 Epoch 20/100
 251/251 1297s 5s/step -
 binary_accuracy: 0.8770 - loss: 0.2939 - val_binary_accuracy: 0.8710 - val_loss:
 0.2985
 Epoch 21/100
 251/251 1306s 5s/step -
 binary_accuracy: 0.8794 - loss: 0.2892 - val_binary_accuracy: 0.8757 - val_loss:
 0.3060
 Epoch 22/100
 251/251 1209s 4s/step -
 binary_accuracy: 0.8802 - loss: 0.2863 - val_binary_accuracy: 0.8734 - val_loss:
 0.2964
 Epoch 23/100
 251/251 1238s 4s/step -
 binary_accuracy: 0.8804 - loss: 0.2854 - val_binary_accuracy: 0.8739 - val_loss:
 0.3004
 Epoch 24/100
 251/251 1233s 4s/step -
 binary_accuracy: 0.8814 - loss: 0.2839 - val_binary_accuracy: 0.8752 - val_loss:
 0.3024
 Epoch 25/100
 251/251 1268s 4s/step -
 binary_accuracy: 0.8829 - loss: 0.2799 - val_binary_accuracy: 0.8756 - val_loss:
 0.2921
 Epoch 26/100
 251/251 1199s 4s/step -
 binary_accuracy: 0.8836 - loss: 0.2777 - val_binary_accuracy: 0.8772 - val_loss:
 0.2984
 Epoch 27/100
 251/251 1234s 4s/step -
 binary_accuracy: 0.8846 - loss: 0.2766 - val_binary_accuracy: 0.8784 - val_loss:
 0.2884
 Epoch 28/100
 251/251 1132s 4s/step -
 binary_accuracy: 0.8840 - loss: 0.2770 - val_binary_accuracy: 0.8773 - val_loss:
 0.2901
 Epoch 29/100
 251/251 1304s 5s/step -
 binary_accuracy: 0.8858 - loss: 0.2721 - val_binary_accuracy: 0.8773 - val_loss:
 0.3007
 Epoch 30/100
 251/251 1361s 5s/step -
 binary_accuracy: 0.8865 - loss: 0.2719 - val_binary_accuracy: 0.8806 - val_loss:
 0.2863
 Epoch 31/100

```

251/251          1358s 5s/step -
binary_accuracy: 0.8888 - loss: 0.2688 - val_binary_accuracy: 0.8876 - val_loss:
0.2772
Epoch 32/100
251/251          1312s 5s/step -
binary_accuracy: 0.8891 - loss: 0.2672 - val_binary_accuracy: 0.8801 - val_loss:
0.2994
Epoch 33/100
251/251          1307s 5s/step -
binary_accuracy: 0.8893 - loss: 0.2666 - val_binary_accuracy: 0.8798 - val_loss:
0.2941
Epoch 34/100
251/251          1341s 5s/step -
binary_accuracy: 0.8905 - loss: 0.2624 - val_binary_accuracy: 0.8831 - val_loss:
0.2835
Epoch 35/100
251/251          1330s 5s/step -
binary_accuracy: 0.8914 - loss: 0.2619 - val_binary_accuracy: 0.8813 - val_loss:
0.3016
Epoch 36/100
251/251          1334s 5s/step -
binary_accuracy: 0.8916 - loss: 0.2601 - val_binary_accuracy: 0.8890 - val_loss:
0.2830
Epoch 37/100
251/251          1293s 5s/step -
binary_accuracy: 0.8931 - loss: 0.2577 - val_binary_accuracy: 0.8792 - val_loss:
0.3259
Epoch 38/100
251/251          1185s 4s/step -
binary_accuracy: 0.8935 - loss: 0.2575 - val_binary_accuracy: 0.8839 - val_loss:
0.2839
Epoch 39/100
251/251          1160s 4s/step -
binary_accuracy: 0.8946 - loss: 0.2558 - val_binary_accuracy: 0.8827 - val_loss:
0.2836
Epoch 40/100
251/251          1301s 5s/step -
binary_accuracy: 0.8960 - loss: 0.2528 - val_binary_accuracy: 0.8794 - val_loss:
0.3058
Epoch 41/100
251/251          1272s 4s/step -
binary_accuracy: 0.8956 - loss: 0.2524 - val_binary_accuracy: 0.8797 - val_loss:
0.2973

```

```

[56]: import matplotlib.pyplot as plt
      accuracy = history.history["binary_accuracy"]
      val_accuracy = history.history["val_binary_accuracy"]

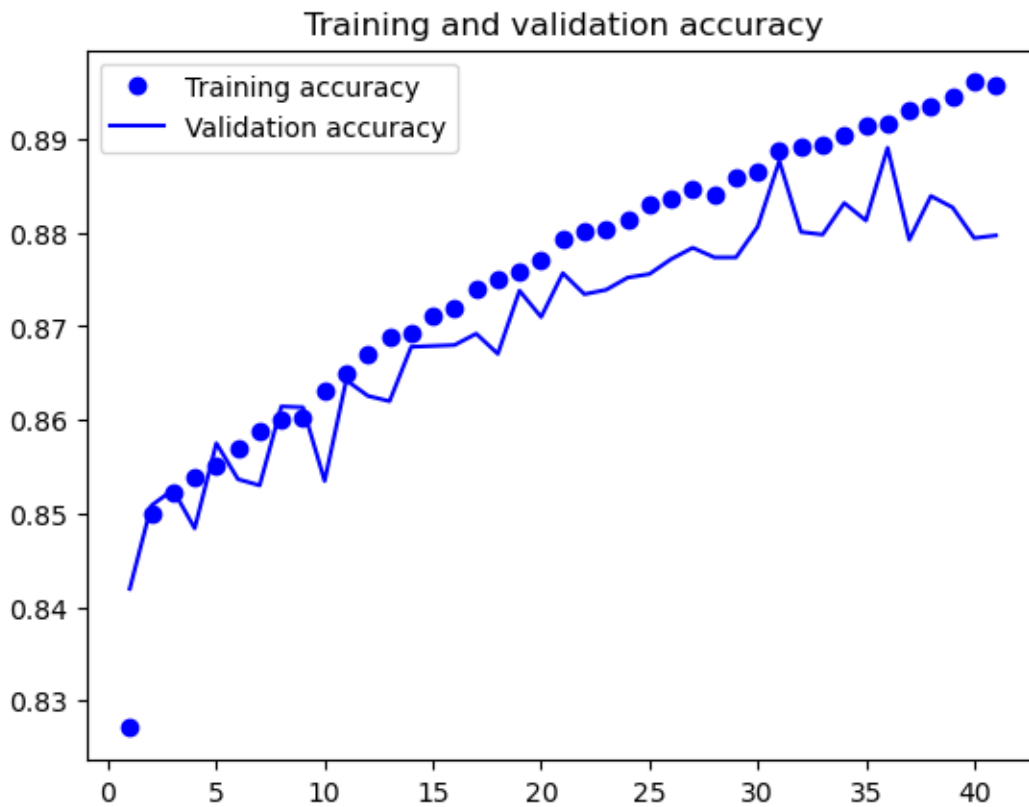
```

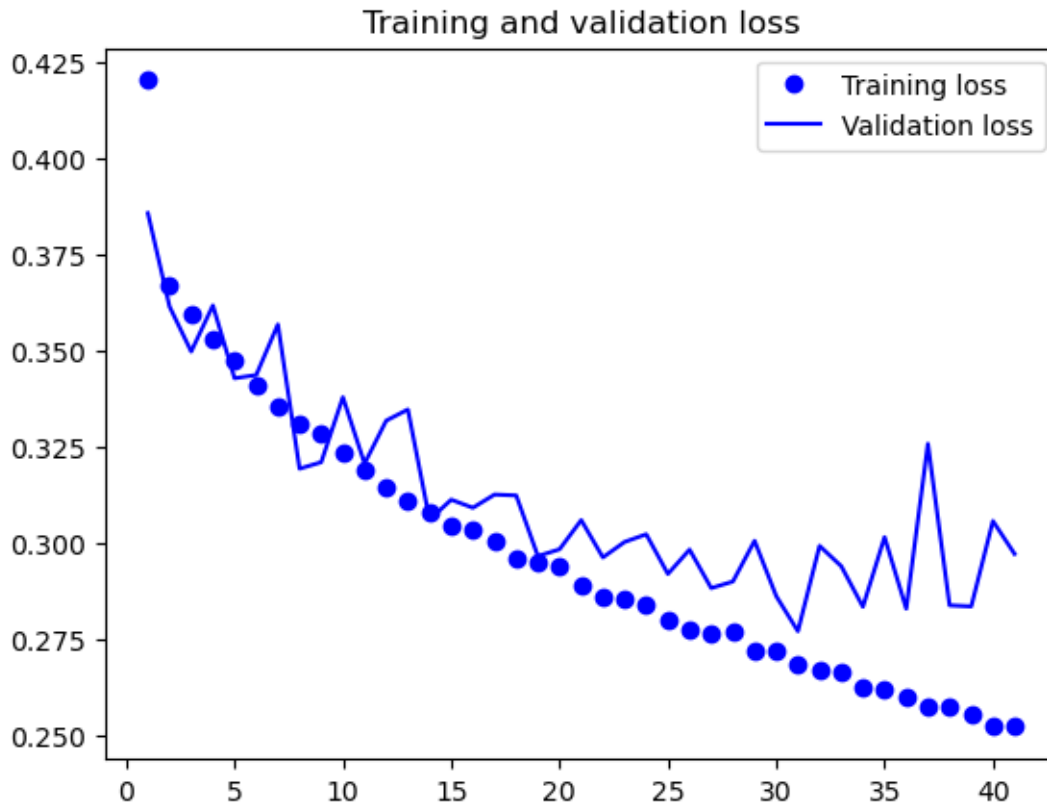


```

loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()

```





```
[60]: test_model = keras.models.load_model(
      "convnet_from_scratch_with_augmentation.keras")
      test_loss, test_acc = test_model.evaluate(test_dataset)
      print(f"Test accuracy: {test_acc:.3f}")
```

```
33/33          126s 4s/step -
binary_accuracy: 0.8684 - loss: 0.3052
Test accuracy: 0.868
```

```
[61]: import numpy as np

      y_pred_probs = test_model.predict(test_dataset)
```

```
33/33          124s 4s/step
```

```
[ ]:
```

```
[69]: THRESHOLD = 0.5
      for images, true_labels in test_dataset.take(1):
          preds = test_model.predict(images)

          for i in range(3):  # inspect first 3 images
```

```

print(f"\n===== Image {i} =====")

# Show image
img = images[i].numpy()          # float32 [0-255]
img = np.clip(img, 0, 255)       # safety
img = img.astype("uint8")        # convert to uint8

plt.figure(figsize=(4,4))
plt.imshow(img)
plt.axis("off")
plt.show()
print(f"{'Label':35s} | {'True':^6s} | {'Prob (%)':^10s} | {'Pred':
↪^6s}")
print("-" * 65)

for j, label in enumerate(label_names):
    true_val = int(true_labels[i][j].numpy())
    prob = preds[i][j] * 100
    pred_val = int(preds[i][j] >= THRESHOLD)

    print(f"{'label':35s} | {'true_val':^6d} | {'prob':^9.2f}% | {'pred_val':
↪^6d}")

```

1/1 0s 173ms/step

===== Image 0 =====



Label		True		Prob (%)		Pred
bridges_any		0		3.29 %		0
buildings_any		1		99.94 %		1
buildings_affected_or_greater		1		55.49 %		1
buildings_minor_or_greater		1		13.78 %		0
debris_any		1		28.60 %		0
flooding_any		0		3.15 %		0
flooding_structures		0		0.45 %		0
roads_any		1		99.68 %		1
roads_damage		0		1.26 %		0
trees_any		1		98.25 %		1
trees_damage		1		29.35 %		0
water_any		0		42.78 %		0

===== Image 1 =====



Label		True		Prob (%)		Pred
bridges_any		0		1.17 %		0
buildings_any		1		99.85 %		1
buildings_affected_or_greater		1		75.35 %		1
buildings_minor_or_greater		1		14.20 %		0
debris_any		1		38.01 %		0
flooding_any		0		0.90 %		0
flooding_structures		0		0.10 %		0
roads_any		1		99.58 %		1

roads_damage	0	0.31	% 0
trees_any	1	92.71	% 1
trees_damage	1	13.73	% 0
water_any	0	34.78	% 0

===== Image 2 =====



Label	True	Prob (%)	Pred
bridges_any	0	9.96	% 0
buildings_any	0	66.15	% 1
buildings_affected_or_greater	0	7.70	% 0
buildings_minior_or_greater	0	0.81	% 0
debris_any	0	2.30	% 0
flooding_any	0	21.22	% 0
flooding_structures	0	3.29	% 0
roads_any	1	87.65	% 1
roads_damage	0	4.41	% 0
trees_any	1	99.82	% 1
trees_damage	1	28.01	% 0
water_any	1	92.34	% 1

```
[71]: img = images[1].numpy()      # float32 [0-255]
      img = np.clip(img, 0, 255)  # safety
      img = img.astype("uint8")   # convert to uint8

      plt.figure(figsize=(4,4))
```

```
plt.imshow(img)
plt.axis("off")
plt.show()
```



```
[85]: sample = ds["test"][42]

# PIL image
pil_img = sample["image"]
display(pil_img)
y_true = [sample[l] for l in label_names]
print("True labels:", y_true)
```




True labels: [False, True, True, True, True, False, False, True, False, True, True, False]

```
[86]: IMG_SIZE = (180, 180)
```

```
img = pil_img.resize(IMG_SIZE)
img = np.array(img).astype("float32")    # keep [0-255]
img = np.expand_dims(img, axis=0)
```

```
[88]: pred_probs = test_model.predict(img)[0]
```

1/1 0s 72ms/step

```
[89]: THRESHOLD = 0.5
```

```
print(f"{'Label':35s} | {'True':^8s} | {'Prob (%)':^10s} | {'Pred':^8s}")
print("-" * 70)
```

```
for name, true_val, prob in zip(label_names, y_true, pred_probs):
    pred_val = prob >= THRESHOLD    # this is already True / False
```

```
print(
    f"{name:35s} | "
    f"{str(true_val):^8s} | "
```

```

        f"{prob*100:~9.2f}% | "
        f"{str(pred_val):~8s}"
    )

```

Label		True		Prob (%)		Pred

bridges_any		False		10.60 %		False
buildings_any		True		99.00 %		True
buildings_affected_or_greater		True		51.06 %		True
buildings_minor_or_greater		True		18.53 %		False
debris_any		True		27.16 %		False
flooding_any		False		4.73 %		False
flooding_structures		False		1.04 %		False
roads_any		True		98.96 %		True
roads_damage		False		3.38 %		False
trees_any		True		95.37 %		True
trees_damage		True		38.79 %		False
water_any		False		38.09 %		False

[]:

4 3.Pretrained Model - ResNetV2

```

[90]: IMG_SIZE = (180, 180)
      NUM_LABELS = 12

      conv_base = keras.applications.ResNet50V2(
          weights="imagenet",
          include_top=False,
          input_shape=(180, 180, 3)
      )

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5
94668760/94668760 3s
0us/step

[91]: conv_base.summary()

Model: "resnet50v2"

Layer (type)	Output Shape	Param #
Connected to		
input_layer_3 (InputLayer)	(None, 180, 180, 3)	0 -

conv1_pad (ZeroPadding2D) ↳input_layer_3[0][0]	(None, 186, 186, 3)	0	┐
conv1_conv (Conv2D) ↳conv1_pad[0][0]	(None, 90, 90, 64)	9,472	┐
pool1_pad (ZeroPadding2D) ↳conv1_conv[0][0]	(None, 92, 92, 64)	0	┐
pool1_pool (MaxPooling2D) ↳pool1_pad[0][0]	(None, 45, 45, 64)	0	┐
conv2_block1_preact_bn ↳pool1_pool[0][0] (BatchNormalization) ↳	(None, 45, 45, 64)	256	┐
conv2_block1_preact_relu ↳conv2_block1_preact_bn[0]... (Activation) ↳	(None, 45, 45, 64)	0	┐
conv2_block1_1_conv (Conv2D) ↳conv2_block1_preact_relu[...]	(None, 45, 45, 64)	4,096	┐
conv2_block1_1_bn ↳conv2_block1_1_conv[0][0] (BatchNormalization) ↳	(None, 45, 45, 64)	256	┐
conv2_block1_1_relu ↳conv2_block1_1_bn[0][0] (Activation) ↳	(None, 45, 45, 64)	0	┐
conv2_block1_2_pad ↳conv2_block1_1_relu[0][0] (ZeroPadding2D) ↳	(None, 47, 47, 64)	0	┐
conv2_block1_2_conv (Conv2D) ↳conv2_block1_2_pad[0][0]	(None, 45, 45, 64)	36,864	┐
conv2_block1_2_bn ↳conv2_block1_2_conv[0][0]	(None, 45, 45, 64)	256	┐

```

(BatchNormalization)
↳

conv2_block1_2_relu      (None, 45, 45, 64)      0
↳conv2_block1_2_bn[0][0]
(Activation)
↳

conv2_block1_0_conv (Conv2D)      (None, 45, 45, 256)      16,640
↳conv2_block1_preact_relu[...]

conv2_block1_3_conv (Conv2D)      (None, 45, 45, 256)      16,640
↳conv2_block1_2_relu[0][0]

conv2_block1_out (Add)      (None, 45, 45, 256)      0
↳conv2_block1_0_conv[0][0],
↳conv2_block1_3_conv[0][0]

conv2_block2_preact_bn      (None, 45, 45, 256)      1,024
↳conv2_block1_out[0][0]
(BatchNormalization)
↳

conv2_block2_preact_relu      (None, 45, 45, 256)      0
↳conv2_block2_preact_bn[0]...
(Activation)
↳

conv2_block2_1_conv (Conv2D)      (None, 45, 45, 64)      16,384
↳conv2_block2_preact_relu[...]

conv2_block2_1_bn      (None, 45, 45, 64)      256
↳conv2_block2_1_conv[0][0]
(BatchNormalization)
↳

conv2_block2_1_relu      (None, 45, 45, 64)      0
↳conv2_block2_1_bn[0][0]
(Activation)
↳

conv2_block2_2_pad      (None, 47, 47, 64)      0
↳conv2_block2_1_relu[0][0]
(ZeroPadding2D)
↳

```

conv2_block2_2_conv (Conv2D)	(None, 45, 45, 64)	36,864	┐
↳ conv2_block2_2_pad[0][0]			
conv2_block2_2_bn	(None, 45, 45, 64)	256	┐
↳ conv2_block2_2_conv[0][0]			
(BatchNormalization)			┐
↳			
conv2_block2_2_relu	(None, 45, 45, 64)	0	┐
↳ conv2_block2_2_bn[0][0]			
(Activation)			┐
↳			
conv2_block2_3_conv (Conv2D)	(None, 45, 45, 256)	16,640	┐
↳ conv2_block2_2_relu[0][0]			
conv2_block2_out (Add)	(None, 45, 45, 256)	0	┐
↳ conv2_block1_out[0][0],			
			┐
↳ conv2_block2_3_conv[0][0]			
conv2_block3_preact_bn	(None, 45, 45, 256)	1,024	┐
↳ conv2_block2_out[0][0]			
(BatchNormalization)			┐
↳			
conv2_block3_preact_relu	(None, 45, 45, 256)	0	┐
↳ conv2_block3_preact_bn[0]...			
(Activation)			┐
↳			
conv2_block3_1_conv (Conv2D)	(None, 45, 45, 64)	16,384	┐
↳ conv2_block3_preact_relu[...			
conv2_block3_1_bn	(None, 45, 45, 64)	256	┐
↳ conv2_block3_1_conv[0][0]			
(BatchNormalization)			┐
↳			
conv2_block3_1_relu	(None, 45, 45, 64)	0	┐
↳ conv2_block3_1_bn[0][0]			
(Activation)			┐
↳			

conv2_block3_2_pad	(None, 47, 47, 64)	0	└
↳conv2_block3_1_relu[0][0]			
(ZeroPadding2D)			└
↳			
conv2_block3_2_conv (Conv2D)	(None, 23, 23, 64)	36,864	└
↳conv2_block3_2_pad[0][0]			
conv2_block3_2_bn	(None, 23, 23, 64)	256	└
↳conv2_block3_2_conv[0][0]			
(BatchNormalization)			└
↳			
conv2_block3_2_relu	(None, 23, 23, 64)	0	└
↳conv2_block3_2_bn[0][0]			
(Activation)			└
↳			
max_pooling2d_8	(None, 23, 23, 256)	0	└
↳conv2_block2_out[0][0]			
(MaxPooling2D)			└
↳			
conv2_block3_3_conv (Conv2D)	(None, 23, 23, 256)	16,640	└
↳conv2_block3_2_relu[0][0]			
conv2_block3_out (Add)	(None, 23, 23, 256)	0	└
↳max_pooling2d_8[0][0],			
			└
↳conv2_block3_3_conv[0][0]			
conv3_block1_preact_bn	(None, 23, 23, 256)	1,024	└
↳conv2_block3_out[0][0]			
(BatchNormalization)			└
↳			
conv3_block1_preact_relu	(None, 23, 23, 256)	0	└
↳conv3_block1_preact_bn[0]...			
(Activation)			└
↳			
conv3_block1_1_conv (Conv2D)	(None, 23, 23, 128)	32,768	└
↳conv3_block1_preact_relu[...			
conv3_block1_1_bn	(None, 23, 23, 128)	512	└
↳conv3_block1_1_conv[0][0]			

(BatchNormalization)			□
↪			
conv3_block1_1_relu	(None, 23, 23, 128)	0	□
↪conv3_block1_1_bn[0][0]			
(Activation)			□
↪			
conv3_block1_2_pad	(None, 25, 25, 128)	0	□
↪conv3_block1_1_relu[0][0]			
(ZeroPadding2D)			□
↪			
conv3_block1_2_conv (Conv2D)	(None, 23, 23, 128)	147,456	□
↪conv3_block1_2_pad[0][0]			
conv3_block1_2_bn	(None, 23, 23, 128)	512	□
↪conv3_block1_2_conv[0][0]			
(BatchNormalization)			□
↪			
conv3_block1_2_relu	(None, 23, 23, 128)	0	□
↪conv3_block1_2_bn[0][0]			
(Activation)			□
↪			
conv3_block1_0_conv (Conv2D)	(None, 23, 23, 512)	131,584	□
↪conv3_block1_preact_relu[...			
conv3_block1_3_conv (Conv2D)	(None, 23, 23, 512)	66,048	□
↪conv3_block1_2_relu[0][0]			
conv3_block1_out (Add)	(None, 23, 23, 512)	0	□
↪conv3_block1_0_conv[0][0],			
			□
↪conv3_block1_3_conv[0][0]			
conv3_block2_preact_bn	(None, 23, 23, 512)	2,048	□
↪conv3_block1_out[0][0]			
(BatchNormalization)			□
↪			
conv3_block2_preact_relu	(None, 23, 23, 512)	0	□
↪conv3_block2_preact_bn[0]...			
(Activation)			□
↪			

conv3_block2_1_conv (Conv2D)	(None, 23, 23, 128)	65,536	┐
↳conv3_block2_preact_relu[...			
conv3_block2_1_bn	(None, 23, 23, 128)	512	┐
↳conv3_block2_1_conv[0][0]			
(BatchNormalization)			┐
↳			
conv3_block2_1_relu	(None, 23, 23, 128)	0	┐
↳conv3_block2_1_bn[0][0]			
(Activation)			┐
↳			
conv3_block2_2_pad	(None, 25, 25, 128)	0	┐
↳conv3_block2_1_relu[0][0]			
(ZeroPadding2D)			┐
↳			
conv3_block2_2_conv (Conv2D)	(None, 23, 23, 128)	147,456	┐
↳conv3_block2_2_pad[0][0]			
conv3_block2_2_bn	(None, 23, 23, 128)	512	┐
↳conv3_block2_2_conv[0][0]			
(BatchNormalization)			┐
↳			
conv3_block2_2_relu	(None, 23, 23, 128)	0	┐
↳conv3_block2_2_bn[0][0]			
(Activation)			┐
↳			
conv3_block2_3_conv (Conv2D)	(None, 23, 23, 512)	66,048	┐
↳conv3_block2_2_relu[0][0]			
conv3_block2_out (Add)	(None, 23, 23, 512)	0	┐
↳conv3_block1_out[0][0],			
			┐
↳conv3_block2_3_conv[0][0]			
conv3_block3_preact_bn	(None, 23, 23, 512)	2,048	┐
↳conv3_block2_out[0][0]			
(BatchNormalization)			┐
↳			

conv3_block3_preact_relu	(None, 23, 23, 512)	0	┐
↳conv3_block3_preact_bn[0]...			
(Activation)			┐
↳			
conv3_block3_1_conv (Conv2D)	(None, 23, 23, 128)	65,536	┐
↳conv3_block3_preact_relu[...			
conv3_block3_1_bn	(None, 23, 23, 128)	512	┐
↳conv3_block3_1_conv[0][0]			
(BatchNormalization)			┐
↳			
conv3_block3_1_relu	(None, 23, 23, 128)	0	┐
↳conv3_block3_1_bn[0][0]			
(Activation)			┐
↳			
conv3_block3_2_pad	(None, 25, 25, 128)	0	┐
↳conv3_block3_1_relu[0][0]			
(ZeroPadding2D)			┐
↳			
conv3_block3_2_conv (Conv2D)	(None, 23, 23, 128)	147,456	┐
↳conv3_block3_2_pad[0][0]			
conv3_block3_2_bn	(None, 23, 23, 128)	512	┐
↳conv3_block3_2_conv[0][0]			
(BatchNormalization)			┐
↳			
conv3_block3_2_relu	(None, 23, 23, 128)	0	┐
↳conv3_block3_2_bn[0][0]			
(Activation)			┐
↳			
conv3_block3_3_conv (Conv2D)	(None, 23, 23, 512)	66,048	┐
↳conv3_block3_2_relu[0][0]			
conv3_block3_out (Add)	(None, 23, 23, 512)	0	┐
↳conv3_block2_out[0][0],			
			┐
↳conv3_block3_3_conv[0][0]			
conv3_block4_preact_bn	(None, 23, 23, 512)	2,048	┐
↳conv3_block3_out[0][0]			

(BatchNormalization)			□
↪			
conv3_block4_preact_relu	(None, 23, 23, 512)	0	□
↪conv3_block4_preact_bn[0]...			
(Activation)			□
↪			
conv3_block4_1_conv (Conv2D)	(None, 23, 23, 128)	65,536	□
↪conv3_block4_preact_relu[...			
conv3_block4_1_bn	(None, 23, 23, 128)	512	□
↪conv3_block4_1_conv[0][0]			
(BatchNormalization)			□
↪			
conv3_block4_1_relu	(None, 23, 23, 128)	0	□
↪conv3_block4_1_bn[0][0]			
(Activation)			□
↪			
conv3_block4_2_pad	(None, 25, 25, 128)	0	□
↪conv3_block4_1_relu[0][0]			
(ZeroPadding2D)			□
↪			
conv3_block4_2_conv (Conv2D)	(None, 12, 12, 128)	147,456	□
↪conv3_block4_2_pad[0][0]			
conv3_block4_2_bn	(None, 12, 12, 128)	512	□
↪conv3_block4_2_conv[0][0]			
(BatchNormalization)			□
↪			
conv3_block4_2_relu	(None, 12, 12, 128)	0	□
↪conv3_block4_2_bn[0][0]			
(Activation)			□
↪			
max_pooling2d_9	(None, 12, 12, 512)	0	□
↪conv3_block3_out[0][0]			
(MaxPooling2D)			□
↪			
conv3_block4_3_conv (Conv2D)	(None, 12, 12, 512)	66,048	□
↪conv3_block4_2_relu[0][0]			

conv3_block4_out (Add)	(None, 12, 12, 512)	0	┐
↳ max_pooling2d_9[0][0],			┐
↳ conv3_block4_3_conv[0][0]			┐
conv4_block1_preact_bn	(None, 12, 12, 512)	2,048	┐
↳ conv3_block4_out[0][0]			┐
(BatchNormalization)			┐
↳			
conv4_block1_preact_relu	(None, 12, 12, 512)	0	┐
↳ conv4_block1_preact_bn[0]...			┐
(Activation)			┐
↳			
conv4_block1_1_conv (Conv2D)	(None, 12, 12, 256)	131,072	┐
↳ conv4_block1_preact_relu[...			
conv4_block1_1_bn	(None, 12, 12, 256)	1,024	┐
↳ conv4_block1_1_conv[0][0]			┐
(BatchNormalization)			┐
↳			
conv4_block1_1_relu	(None, 12, 12, 256)	0	┐
↳ conv4_block1_1_bn[0][0]			┐
(Activation)			┐
↳			
conv4_block1_2_pad	(None, 14, 14, 256)	0	┐
↳ conv4_block1_1_relu[0][0]			┐
(ZeroPadding2D)			┐
↳			
conv4_block1_2_conv (Conv2D)	(None, 12, 12, 256)	589,824	┐
↳ conv4_block1_2_pad[0][0]			
conv4_block1_2_bn	(None, 12, 12, 256)	1,024	┐
↳ conv4_block1_2_conv[0][0]			┐
(BatchNormalization)			┐
↳			
conv4_block1_2_relu	(None, 12, 12, 256)	0	┐
↳ conv4_block1_2_bn[0][0]			┐
(Activation)			┐
↳			

conv4_block1_0_conv (Conv2D)	(None, 12, 12, 1024)	525,312	┐
↳conv4_block1_preact_relu[...			
conv4_block1_3_conv (Conv2D)	(None, 12, 12, 1024)	263,168	┐
↳conv4_block1_2_relu[0][0]			
conv4_block1_out (Add)	(None, 12, 12, 1024)	0	┐
↳conv4_block1_0_conv[0][0],			
			┐
↳conv4_block1_3_conv[0][0]			
conv4_block2_preact_bn	(None, 12, 12, 1024)	4,096	┐
↳conv4_block1_out[0][0]			
(BatchNormalization)			┐
↳			
conv4_block2_preact_relu	(None, 12, 12, 1024)	0	┐
↳conv4_block2_preact_bn[0]...			
(Activation)			┐
↳			
conv4_block2_1_conv (Conv2D)	(None, 12, 12, 256)	262,144	┐
↳conv4_block2_preact_relu[...			
conv4_block2_1_bn	(None, 12, 12, 256)	1,024	┐
↳conv4_block2_1_conv[0][0]			
(BatchNormalization)			┐
↳			
conv4_block2_1_relu	(None, 12, 12, 256)	0	┐
↳conv4_block2_1_bn[0][0]			
(Activation)			┐
↳			
conv4_block2_2_pad	(None, 14, 14, 256)	0	┐
↳conv4_block2_1_relu[0][0]			
(ZeroPadding2D)			┐
↳			
conv4_block2_2_conv (Conv2D)	(None, 12, 12, 256)	589,824	┐
↳conv4_block2_2_pad[0][0]			
conv4_block2_2_bn	(None, 12, 12, 256)	1,024	┐
↳conv4_block2_2_conv[0][0]			

(BatchNormalization)			□
↪			
conv4_block2_2_relu	(None, 12, 12, 256)	0	□
↪conv4_block2_2_bn[0][0]			
(Activation)			□
↪			
conv4_block2_3_conv (Conv2D)	(None, 12, 12, 1024)	263,168	□
↪conv4_block2_2_relu[0][0]			
conv4_block2_out (Add)	(None, 12, 12, 1024)	0	□
↪conv4_block1_out[0][0],			
			□
↪conv4_block2_3_conv[0][0]			
conv4_block3_preact_bn	(None, 12, 12, 1024)	4,096	□
↪conv4_block2_out[0][0]			
(BatchNormalization)			□
↪			
conv4_block3_preact_relu	(None, 12, 12, 1024)	0	□
↪conv4_block3_preact_bn[0]...			
(Activation)			□
↪			
conv4_block3_1_conv (Conv2D)	(None, 12, 12, 256)	262,144	□
↪conv4_block3_preact_relu[...			
conv4_block3_1_bn	(None, 12, 12, 256)	1,024	□
↪conv4_block3_1_conv[0][0]			
(BatchNormalization)			□
↪			
conv4_block3_1_relu	(None, 12, 12, 256)	0	□
↪conv4_block3_1_bn[0][0]			
(Activation)			□
↪			
conv4_block3_2_pad	(None, 14, 14, 256)	0	□
↪conv4_block3_1_relu[0][0]			
(ZeroPadding2D)			□
↪			
conv4_block3_2_conv (Conv2D)	(None, 12, 12, 256)	589,824	□
↪conv4_block3_2_pad[0][0]			

conv4_block3_2_bn ↳conv4_block3_2_conv[0][0] (BatchNormalization)	(None, 12, 12, 256)	1,024	↳
conv4_block3_2_relu ↳conv4_block3_2_bn[0][0] (Activation)	(None, 12, 12, 256)	0	↳
conv4_block3_3_conv (Conv2D) ↳conv4_block3_2_relu[0][0]	(None, 12, 12, 1024)	263,168	↳
conv4_block3_out (Add) ↳conv4_block2_out[0][0], ↳conv4_block3_3_conv[0][0]	(None, 12, 12, 1024)	0	↳
conv4_block4_preact_bn ↳conv4_block3_out[0][0] (BatchNormalization)	(None, 12, 12, 1024)	4,096	↳
conv4_block4_preact_relu ↳conv4_block4_preact_bn[0]... (Activation)	(None, 12, 12, 1024)	0	↳
conv4_block4_1_conv (Conv2D) ↳conv4_block4_preact_relu[...]	(None, 12, 12, 256)	262,144	↳
conv4_block4_1_bn ↳conv4_block4_1_conv[0][0] (BatchNormalization)	(None, 12, 12, 256)	1,024	↳
conv4_block4_1_relu ↳conv4_block4_1_bn[0][0] (Activation)	(None, 12, 12, 256)	0	↳
conv4_block4_2_pad ↳conv4_block4_1_relu[0][0] (ZeroPadding2D)	(None, 14, 14, 256)	0	↳

conv4_block4_2_conv (Conv2D) ↳conv4_block4_2_pad[0][0]	(None, 12, 12, 256)	589,824	┐
conv4_block4_2_bn ↳conv4_block4_2_conv[0][0] (BatchNormalization)	(None, 12, 12, 256)	1,024	┐
conv4_block4_2_relu ↳conv4_block4_2_bn[0][0] (Activation)	(None, 12, 12, 256)	0	┐
conv4_block4_3_conv (Conv2D) ↳conv4_block4_2_relu[0][0]	(None, 12, 12, 1024)	263,168	┐
conv4_block4_out (Add) ↳conv4_block3_out[0][0], ↳conv4_block4_3_conv[0][0]	(None, 12, 12, 1024)	0	┐
conv4_block5_preact_bn ↳conv4_block4_out[0][0] (BatchNormalization)	(None, 12, 12, 1024)	4,096	┐
conv4_block5_preact_relu ↳conv4_block5_preact_bn[0]... (Activation)	(None, 12, 12, 1024)	0	┐
conv4_block5_1_conv (Conv2D) ↳conv4_block5_preact_relu[...]	(None, 12, 12, 256)	262,144	┐
conv4_block5_1_bn ↳conv4_block5_1_conv[0][0] (BatchNormalization)	(None, 12, 12, 256)	1,024	┐
conv4_block5_1_relu ↳conv4_block5_1_bn[0][0] (Activation)	(None, 12, 12, 256)	0	┐

conv4_block5_2_pad	(None, 14, 14, 256)	0	┐
↳conv4_block5_1_relu[0][0]			
(ZeroPadding2D)			┐
↳			
conv4_block5_2_conv (Conv2D)	(None, 12, 12, 256)	589,824	┐
↳conv4_block5_2_pad[0][0]			
conv4_block5_2_bn	(None, 12, 12, 256)	1,024	┐
↳conv4_block5_2_conv[0][0]			
(BatchNormalization)			┐
↳			
conv4_block5_2_relu	(None, 12, 12, 256)	0	┐
↳conv4_block5_2_bn[0][0]			
(Activation)			┐
↳			
conv4_block5_3_conv (Conv2D)	(None, 12, 12, 1024)	263,168	┐
↳conv4_block5_2_relu[0][0]			
conv4_block5_out (Add)	(None, 12, 12, 1024)	0	┐
↳conv4_block4_out[0][0],			
			┐
↳conv4_block5_3_conv[0][0]			
conv4_block6_preact_bn	(None, 12, 12, 1024)	4,096	┐
↳conv4_block5_out[0][0]			
(BatchNormalization)			┐
↳			
conv4_block6_preact_relu	(None, 12, 12, 1024)	0	┐
↳conv4_block6_preact_bn[0]...			
(Activation)			┐
↳			
conv4_block6_1_conv (Conv2D)	(None, 12, 12, 256)	262,144	┐
↳conv4_block6_preact_relu[...			
conv4_block6_1_bn	(None, 12, 12, 256)	1,024	┐
↳conv4_block6_1_conv[0][0]			
(BatchNormalization)			┐
↳			
conv4_block6_1_relu	(None, 12, 12, 256)	0	┐
↳conv4_block6_1_bn[0][0]			

(Activation)			□
↪			
conv4_block6_2_pad	(None, 14, 14, 256)	0	□
↪conv4_block6_1_relu[0][0]			
(ZeroPadding2D)			□
↪			
conv4_block6_2_conv (Conv2D)	(None, 6, 6, 256)	589,824	□
↪conv4_block6_2_pad[0][0]			
conv4_block6_2_bn	(None, 6, 6, 256)	1,024	□
↪conv4_block6_2_conv[0][0]			
(BatchNormalization)			□
↪			
conv4_block6_2_relu	(None, 6, 6, 256)	0	□
↪conv4_block6_2_bn[0][0]			
(Activation)			□
↪			
max_pooling2d_10	(None, 6, 6, 1024)	0	□
↪conv4_block5_out[0][0]			
(MaxPooling2D)			□
↪			
conv4_block6_3_conv (Conv2D)	(None, 6, 6, 1024)	263,168	□
↪conv4_block6_2_relu[0][0]			
conv4_block6_out (Add)	(None, 6, 6, 1024)	0	□
↪max_pooling2d_10[0][0],			
			□
↪conv4_block6_3_conv[0][0]			
conv5_block1_preact_bn	(None, 6, 6, 1024)	4,096	□
↪conv4_block6_out[0][0]			
(BatchNormalization)			□
↪			
conv5_block1_preact_relu	(None, 6, 6, 1024)	0	□
↪conv5_block1_preact_bn[0]...			
(Activation)			□
↪			
conv5_block1_1_conv (Conv2D)	(None, 6, 6, 512)	524,288	□
↪conv5_block1_preact_relu[...			

conv5_block1_1_bn	(None, 6, 6, 512)	2,048	┐
↳conv5_block1_1_conv[0][0]			
(BatchNormalization)			┐
↳			
conv5_block1_1_relu	(None, 6, 6, 512)	0	┐
↳conv5_block1_1_bn[0][0]			
(Activation)			┐
↳			
conv5_block1_2_pad	(None, 8, 8, 512)	0	┐
↳conv5_block1_1_relu[0][0]			
(ZeroPadding2D)			┐
↳			
conv5_block1_2_conv (Conv2D)	(None, 6, 6, 512)	2,359,296	┐
↳conv5_block1_2_pad[0][0]			
conv5_block1_2_bn	(None, 6, 6, 512)	2,048	┐
↳conv5_block1_2_conv[0][0]			
(BatchNormalization)			┐
↳			
conv5_block1_2_relu	(None, 6, 6, 512)	0	┐
↳conv5_block1_2_bn[0][0]			
(Activation)			┐
↳			
conv5_block1_0_conv (Conv2D)	(None, 6, 6, 2048)	2,099,200	┐
↳conv5_block1_preact_relu[...]			
conv5_block1_3_conv (Conv2D)	(None, 6, 6, 2048)	1,050,624	┐
↳conv5_block1_2_relu[0][0]			
conv5_block1_out (Add)	(None, 6, 6, 2048)	0	┐
↳conv5_block1_0_conv[0][0],			
			┐
↳conv5_block1_3_conv[0][0]			
conv5_block2_preact_bn	(None, 6, 6, 2048)	8,192	┐
↳conv5_block1_out[0][0]			
(BatchNormalization)			┐
↳			

conv5_block2_preact_relu	(None, 6, 6, 2048)	0	┘
↳conv5_block2_preact_bn[0]...			
(Activation)			┘
↳			
conv5_block2_1_conv (Conv2D)	(None, 6, 6, 512)	1,048,576	┘
↳conv5_block2_preact_relu[...			
conv5_block2_1_bn	(None, 6, 6, 512)	2,048	┘
↳conv5_block2_1_conv[0][0]			
(BatchNormalization)			┘
↳			
conv5_block2_1_relu	(None, 6, 6, 512)	0	┘
↳conv5_block2_1_bn[0][0]			
(Activation)			┘
↳			
conv5_block2_2_pad	(None, 8, 8, 512)	0	┘
↳conv5_block2_1_relu[0][0]			
(ZeroPadding2D)			┘
↳			
conv5_block2_2_conv (Conv2D)	(None, 6, 6, 512)	2,359,296	┘
↳conv5_block2_2_pad[0][0]			
conv5_block2_2_bn	(None, 6, 6, 512)	2,048	┘
↳conv5_block2_2_conv[0][0]			
(BatchNormalization)			┘
↳			
conv5_block2_2_relu	(None, 6, 6, 512)	0	┘
↳conv5_block2_2_bn[0][0]			
(Activation)			┘
↳			
conv5_block2_3_conv (Conv2D)	(None, 6, 6, 2048)	1,050,624	┘
↳conv5_block2_2_relu[0][0]			
conv5_block2_out (Add)	(None, 6, 6, 2048)	0	┘
↳conv5_block1_out[0][0],			
			┘
↳conv5_block2_3_conv[0][0]			
conv5_block3_preact_bn	(None, 6, 6, 2048)	8,192	┘
↳conv5_block2_out[0][0]			

```

(BatchNormalization)
↳

conv5_block3_preact_relu      (None, 6, 6, 2048)      0
↳conv5_block3_preact_bn[0]...
(Activation)
↳

conv5_block3_1_conv (Conv2D)      (None, 6, 6, 512)      1,048,576
↳conv5_block3_preact_relu[...]

conv5_block3_1_bn      (None, 6, 6, 512)      2,048
↳conv5_block3_1_conv[0][0]
(BatchNormalization)
↳

conv5_block3_1_relu      (None, 6, 6, 512)      0
↳conv5_block3_1_bn[0][0]
(Activation)
↳

conv5_block3_2_pad      (None, 8, 8, 512)      0
↳conv5_block3_1_relu[0][0]
(ZeroPadding2D)
↳

conv5_block3_2_conv (Conv2D)      (None, 6, 6, 512)      2,359,296
↳conv5_block3_2_pad[0][0]

conv5_block3_2_bn      (None, 6, 6, 512)      2,048
↳conv5_block3_2_conv[0][0]
(BatchNormalization)
↳

conv5_block3_2_relu      (None, 6, 6, 512)      0
↳conv5_block3_2_bn[0][0]
(Activation)
↳

conv5_block3_3_conv (Conv2D)      (None, 6, 6, 2048)      1,050,624
↳conv5_block3_2_relu[0][0]

conv5_block3_out (Add)      (None, 6, 6, 2048)      0
↳conv5_block2_out[0][0],

↳conv5_block3_3_conv[0][0]

```

```
post_bn (BatchNormalization)      (None, 6, 6, 2048)      8,192 ▮  
↳ conv5_block3_out[0][0]
```

```
post_relu (Activation)            (None, 6, 6, 2048)      0 ▮  
↳ post_bn[0][0]
```

Total params: 23,564,800 (89.89 MB)

Trainable params: 23,519,360 (89.72 MB)

Non-trainable params: 45,440 (177.50 KB)

4.1 a) FAST FEATURE EXTRACTION WITHOUT DATA AUGMENTATION

```
[92]: import numpy as np  
      from tensorflow import keras  
  
      def get_features_and_labels(dataset, conv_base):  
          all_features = []  
          all_labels = []  
  
          for images, labels in dataset:  
              # Correct preprocessing for ResNetV2  
              images = keras.applications.resnet_v2.preprocess_input(images)  
  
              # Extract features  
              features = conv_base(images, training=False)  
  
              # Flatten spatial dimensions: (B, 6, 6, 2048) → (B, 73728)  
              features = keras.layers.GlobalAveragePooling2D()(features)  
  
              all_features.append(features.numpy())  
              all_labels.append(labels.numpy())  
  
          return (  
              np.concatenate(all_features),  
              np.concatenate(all_labels)  
          )
```

```
[93]: train_features, train_labels = get_features_and_labels(train_dataset, conv_base)  
      val_features, val_labels      = get_features_and_labels(validation_dataset, ▮  
                                    ↳ conv_base)
```

```
test_features, test_labels = get_features_and_labels(test_dataset, conv_base)
```

```
[94]: train_features.shape
```

```
[94]: (8030, 2048)
```

```
[96]: train_labels.shape
```

```
[96]: (8030, 12)
```

```
[97]: NUM_LABELS = 12
```

```
# INPUT = extracted features
inputs = keras.Input(shape=(2048,))

x = layers.Dense(512, activation="relu")(inputs)
x = layers.Dropout(0.5)(x)

x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)

# MULTI-LABEL OUTPUT
outputs = layers.Dense(NUM_LABELS, activation="sigmoid")(x)

model = keras.Model(inputs, outputs)

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["binary_accuracy"]
)

model.summary()
```

Model: "functional_6"

Layer (type)	Output Shape	
Param #		
input_layer_4 (InputLayer)	(None, 2048)	
↪ 0		
dense_2 (Dense)	(None, 512)	
↪ 1,049,088		

```

dropout_1 (Dropout)                                (None, 512)
↳ 0

dense_3 (Dense)                                      (None, 256)
↳131,328

dropout_2 (Dropout)                                (None, 256)
↳ 0

dense_4 (Dense)                                      (None, 12)
↳3,084

```

Total params: 1,183,500 (4.51 MB)

Trainable params: 1,183,500 (4.51 MB)

Non-trainable params: 0 (0.00 B)

```

[101]: callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="feature_extraction_resnet50v2.keras",
        save_best_only=True,
        monitor="val_loss"
    ),
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=10,
        restore_best_weights=True
    )
]

```

```

[102]: history = model.fit(
    train_features,
    train_labels,
    epochs=50,
    batch_size=32,
    validation_data=(val_features, val_labels),
    callbacks=callbacks
)

```

Epoch 1/50

251/251 2s 10ms/step -

binary_accuracy: 0.8940 - loss: 0.2559 - val_binary_accuracy: 0.8848 - val_loss: 0.2835

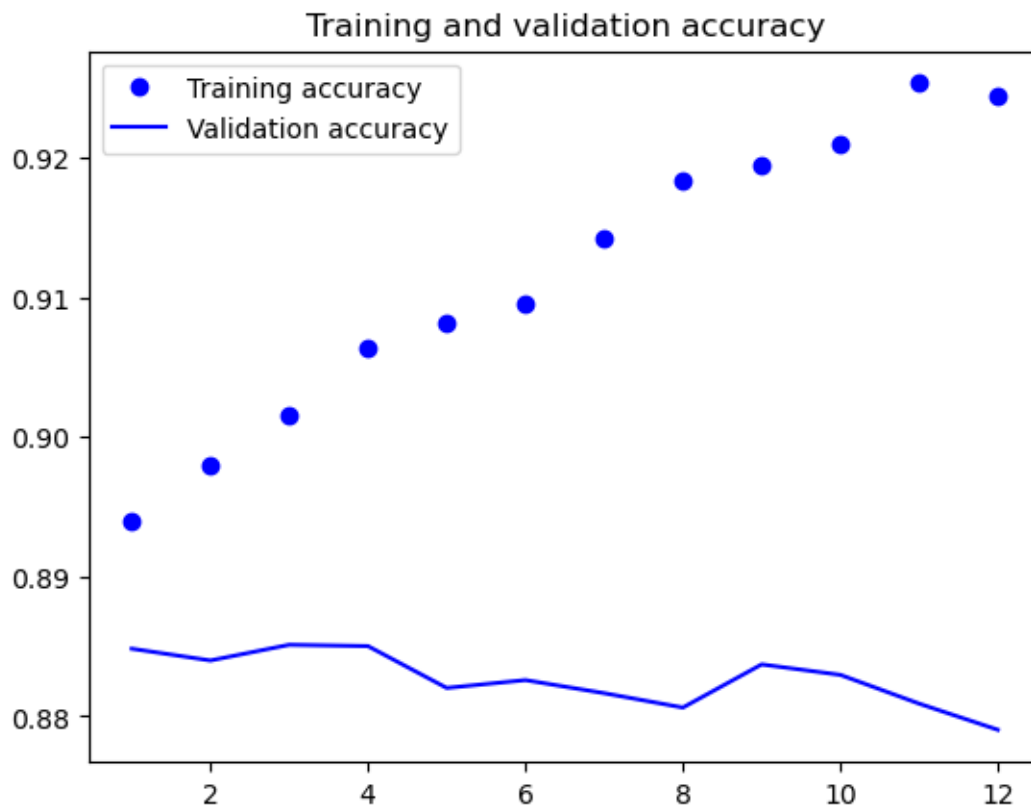
```

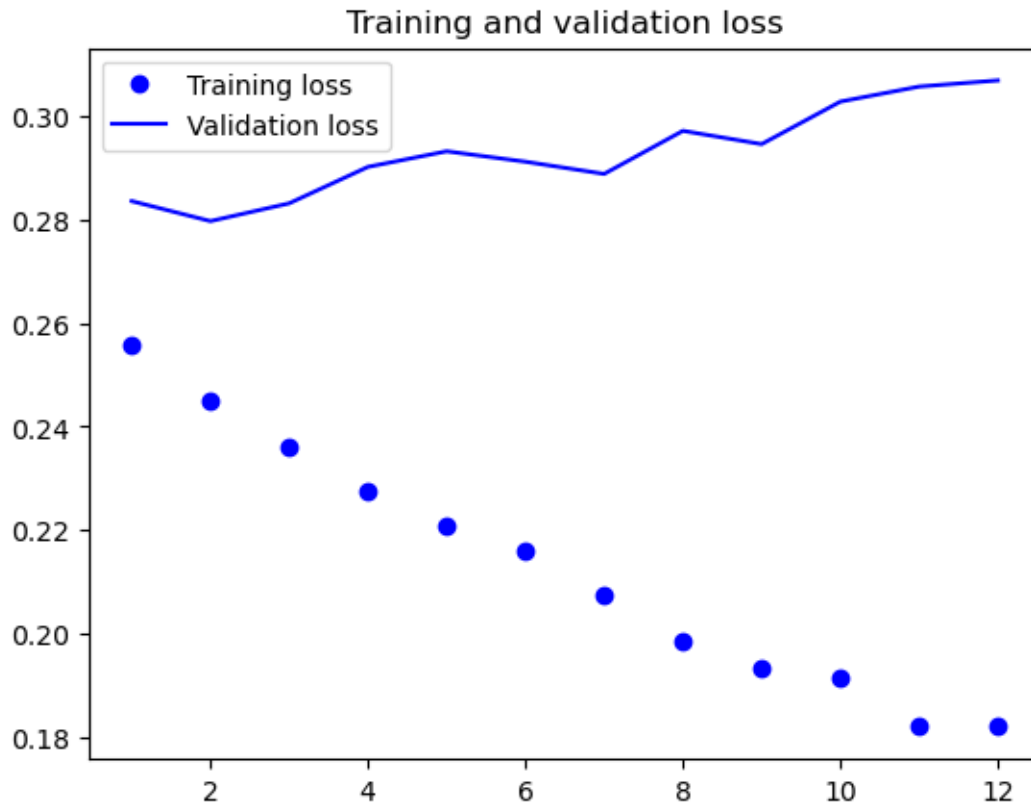
Epoch 2/50
251/251          2s 9ms/step -
binary_accuracy: 0.8979 - loss: 0.2451 - val_binary_accuracy: 0.8840 - val_loss:
0.2797
Epoch 3/50
251/251          2s 9ms/step -
binary_accuracy: 0.9016 - loss: 0.2362 - val_binary_accuracy: 0.8851 - val_loss:
0.2831
Epoch 4/50
251/251          2s 8ms/step -
binary_accuracy: 0.9063 - loss: 0.2275 - val_binary_accuracy: 0.8850 - val_loss:
0.2902
Epoch 5/50
251/251          2s 8ms/step -
binary_accuracy: 0.9082 - loss: 0.2209 - val_binary_accuracy: 0.8820 - val_loss:
0.2932
Epoch 6/50
251/251          3s 8ms/step -
binary_accuracy: 0.9096 - loss: 0.2161 - val_binary_accuracy: 0.8826 - val_loss:
0.2912
Epoch 7/50
251/251          2s 9ms/step -
binary_accuracy: 0.9141 - loss: 0.2073 - val_binary_accuracy: 0.8816 - val_loss:
0.2888
Epoch 8/50
251/251          3s 9ms/step -
binary_accuracy: 0.9184 - loss: 0.1984 - val_binary_accuracy: 0.8806 - val_loss:
0.2971
Epoch 9/50
251/251          2s 8ms/step -
binary_accuracy: 0.9195 - loss: 0.1935 - val_binary_accuracy: 0.8837 - val_loss:
0.2946
Epoch 10/50
251/251          2s 8ms/step -
binary_accuracy: 0.9209 - loss: 0.1916 - val_binary_accuracy: 0.8830 - val_loss:
0.3028
Epoch 11/50
251/251          3s 9ms/step -
binary_accuracy: 0.9253 - loss: 0.1824 - val_binary_accuracy: 0.8809 - val_loss:
0.3057
Epoch 12/50
251/251          3s 9ms/step -
binary_accuracy: 0.9244 - loss: 0.1820 - val_binary_accuracy: 0.8790 - val_loss:
0.3069

```

[]:

```
[103]: accuracy = history.history["binary_accuracy"]
val_accuracy = history.history["val_binary_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





```
[106]: test_model = keras.models.load_model(
        "feature_extraction_resnet50v2.keras")
test_loss, test_acc = test_model.evaluate(
    test_features,
    test_labels,
    batch_size=32
)

print(f"Test accuracy: {test_acc:.3f}")
```

```
33/33          0s 4ms/step -
binary_accuracy: 0.8904 - loss: 0.2823
Test accuracy: 0.890
```

4.2 b) Fine-tuning a pretrained model

```
[107]: conv_base.trainable = True

# Freeze everything first
for layer in conv_base.layers:
    layer.trainable = False
```



```
# Unfreeze only last 3 Conv2D layers
conv_layers = [l for l in conv_base.layers if isinstance(l, keras.layers.
↳Conv2D)]
for layer in conv_layers[-3:]:
    layer.trainable = True
```

```
[112]: model = keras.Sequential([
    keras.Input(shape=(2048,)),
    layers.Dense(512, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(12, activation="sigmoid")
])
```

```
[113]: model.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-5),
    loss="binary_crossentropy",
    metrics=["binary_accuracy"]
)
```

```
[109]: callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="fine_tuning_resnet50v2.keras",
        save_best_only=True,
        monitor="val_loss"
    ),
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=8,
        restore_best_weights=True
    )
]
```

```
[114]: history = model.fit(
    train_features,
    train_labels,
    epochs=50,
    batch_size=32,
    validation_data=(val_features, val_labels),
    callbacks=callbacks
)
```

Epoch 1/50

251/251 4s 10ms/step -

binary_accuracy: 0.7193 - loss: 0.5795 - val_binary_accuracy: 0.8322 - val_loss: 0.4175

Epoch 2/50

251/251 2s 7ms/step -

```

binary_accuracy: 0.8152 - loss: 0.4491 - val_binary_accuracy: 0.8430 - val_loss:
0.3890
Epoch 3/50
251/251          3s 7ms/step -
binary_accuracy: 0.8320 - loss: 0.4207 - val_binary_accuracy: 0.8490 - val_loss:
0.3703
Epoch 4/50
251/251          2s 7ms/step -
binary_accuracy: 0.8380 - loss: 0.4003 - val_binary_accuracy: 0.8541 - val_loss:
0.3589
Epoch 5/50
251/251          2s 7ms/step -
binary_accuracy: 0.8434 - loss: 0.3854 - val_binary_accuracy: 0.8568 - val_loss:
0.3504
Epoch 6/50
251/251          2s 7ms/step -
binary_accuracy: 0.8466 - loss: 0.3762 - val_binary_accuracy: 0.8581 - val_loss:
0.3436
Epoch 7/50
251/251          2s 7ms/step -
binary_accuracy: 0.8513 - loss: 0.3663 - val_binary_accuracy: 0.8599 - val_loss:
0.3384
Epoch 8/50
251/251          2s 7ms/step -
binary_accuracy: 0.8557 - loss: 0.3575 - val_binary_accuracy: 0.8602 - val_loss:
0.3338
Epoch 9/50
251/251          3s 7ms/step -
binary_accuracy: 0.8557 - loss: 0.3530 - val_binary_accuracy: 0.8613 - val_loss:
0.3298
Epoch 10/50
251/251          2s 7ms/step -
binary_accuracy: 0.8578 - loss: 0.3478 - val_binary_accuracy: 0.8626 - val_loss:
0.3264
Epoch 11/50
251/251          3s 7ms/step -
binary_accuracy: 0.8599 - loss: 0.3432 - val_binary_accuracy: 0.8640 - val_loss:
0.3227
Epoch 12/50
251/251          2s 7ms/step -
binary_accuracy: 0.8614 - loss: 0.3392 - val_binary_accuracy: 0.8653 - val_loss:
0.3205
Epoch 13/50
251/251          2s 7ms/step -
binary_accuracy: 0.8629 - loss: 0.3343 - val_binary_accuracy: 0.8653 - val_loss:
0.3180
Epoch 14/50
251/251          3s 7ms/step -

```

```

binary_accuracy: 0.8649 - loss: 0.3297 - val_binary_accuracy: 0.8663 - val_loss:
0.3164
Epoch 15/50
251/251          2s 7ms/step -
binary_accuracy: 0.8651 - loss: 0.3265 - val_binary_accuracy: 0.8667 - val_loss:
0.3145
Epoch 16/50
251/251          3s 7ms/step -
binary_accuracy: 0.8664 - loss: 0.3261 - val_binary_accuracy: 0.8670 - val_loss:
0.3127
Epoch 17/50
251/251          2s 6ms/step -
binary_accuracy: 0.8691 - loss: 0.3209 - val_binary_accuracy: 0.8675 - val_loss:
0.3112
Epoch 18/50
251/251          3s 7ms/step -
binary_accuracy: 0.8694 - loss: 0.3191 - val_binary_accuracy: 0.8680 - val_loss:
0.3095
Epoch 19/50
251/251          2s 7ms/step -
binary_accuracy: 0.8704 - loss: 0.3162 - val_binary_accuracy: 0.8687 - val_loss:
0.3077
Epoch 20/50
251/251          3s 8ms/step -
binary_accuracy: 0.8709 - loss: 0.3139 - val_binary_accuracy: 0.8688 - val_loss:
0.3066
Epoch 21/50
251/251          2s 7ms/step -
binary_accuracy: 0.8712 - loss: 0.3122 - val_binary_accuracy: 0.8696 - val_loss:
0.3054
Epoch 22/50
251/251          2s 7ms/step -
binary_accuracy: 0.8721 - loss: 0.3103 - val_binary_accuracy: 0.8702 - val_loss:
0.3042
Epoch 23/50
251/251          3s 7ms/step -
binary_accuracy: 0.8735 - loss: 0.3066 - val_binary_accuracy: 0.8722 - val_loss:
0.3026
Epoch 24/50
251/251          3s 8ms/step -
binary_accuracy: 0.8749 - loss: 0.3056 - val_binary_accuracy: 0.8725 - val_loss:
0.3019
Epoch 25/50
251/251          2s 7ms/step -
binary_accuracy: 0.8747 - loss: 0.3042 - val_binary_accuracy: 0.8728 - val_loss:
0.3008
Epoch 26/50
251/251          3s 7ms/step -

```

```

binary_accuracy: 0.8762 - loss: 0.3017 - val_binary_accuracy: 0.8732 - val_loss:
0.3002
Epoch 27/50
251/251          3s 7ms/step -
binary_accuracy: 0.8769 - loss: 0.3000 - val_binary_accuracy: 0.8749 - val_loss:
0.2992
Epoch 28/50
251/251          2s 7ms/step -
binary_accuracy: 0.8766 - loss: 0.2994 - val_binary_accuracy: 0.8751 - val_loss:
0.2986
Epoch 29/50
251/251          2s 7ms/step -
binary_accuracy: 0.8777 - loss: 0.2967 - val_binary_accuracy: 0.8754 - val_loss:
0.2976
Epoch 30/50
251/251          2s 8ms/step -
binary_accuracy: 0.8777 - loss: 0.2949 - val_binary_accuracy: 0.8757 - val_loss:
0.2969
Epoch 31/50
251/251          2s 7ms/step -
binary_accuracy: 0.8781 - loss: 0.2939 - val_binary_accuracy: 0.8762 - val_loss:
0.2959
Epoch 32/50
251/251          3s 8ms/step -
binary_accuracy: 0.8801 - loss: 0.2923 - val_binary_accuracy: 0.8764 - val_loss:
0.2956
Epoch 33/50
251/251          2s 7ms/step -
binary_accuracy: 0.8803 - loss: 0.2915 - val_binary_accuracy: 0.8764 - val_loss:
0.2948
Epoch 34/50
251/251          3s 7ms/step -
binary_accuracy: 0.8816 - loss: 0.2890 - val_binary_accuracy: 0.8761 - val_loss:
0.2947
Epoch 35/50
251/251          2s 6ms/step -
binary_accuracy: 0.8821 - loss: 0.2875 - val_binary_accuracy: 0.8767 - val_loss:
0.2937
Epoch 36/50
251/251          2s 7ms/step -
binary_accuracy: 0.8825 - loss: 0.2868 - val_binary_accuracy: 0.8767 - val_loss:
0.2936
Epoch 37/50
251/251          3s 7ms/step -
binary_accuracy: 0.8830 - loss: 0.2850 - val_binary_accuracy: 0.8769 - val_loss:
0.2928
Epoch 38/50
251/251          3s 7ms/step -

```

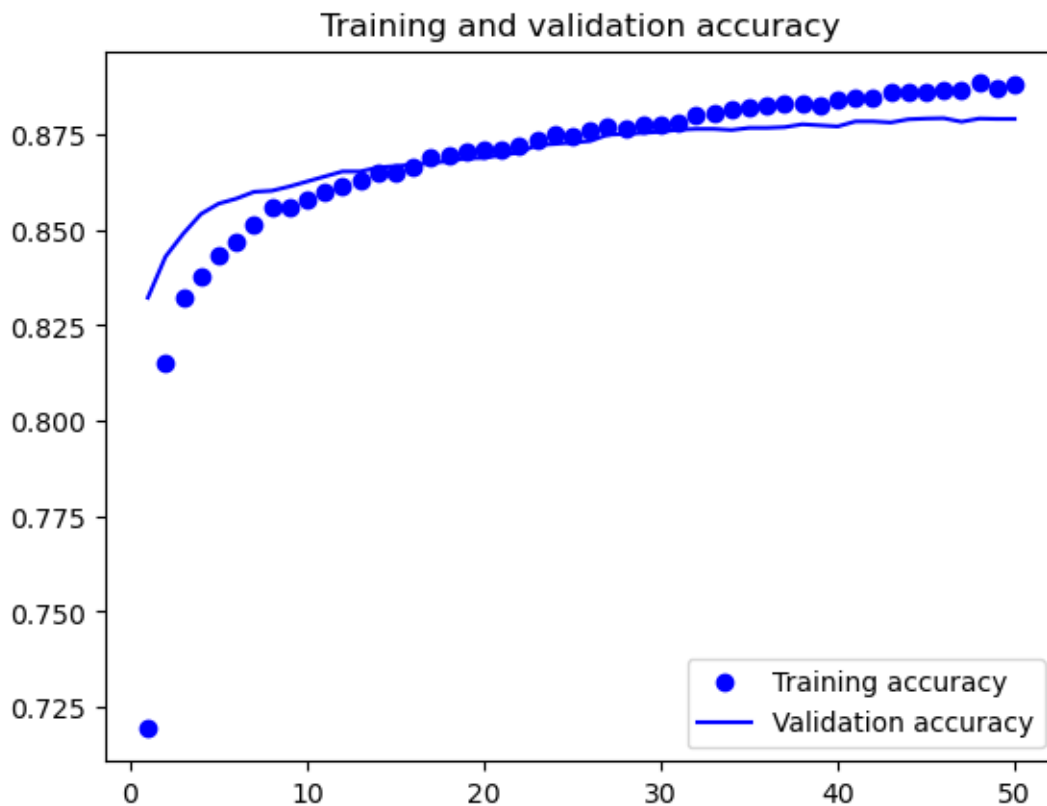
```

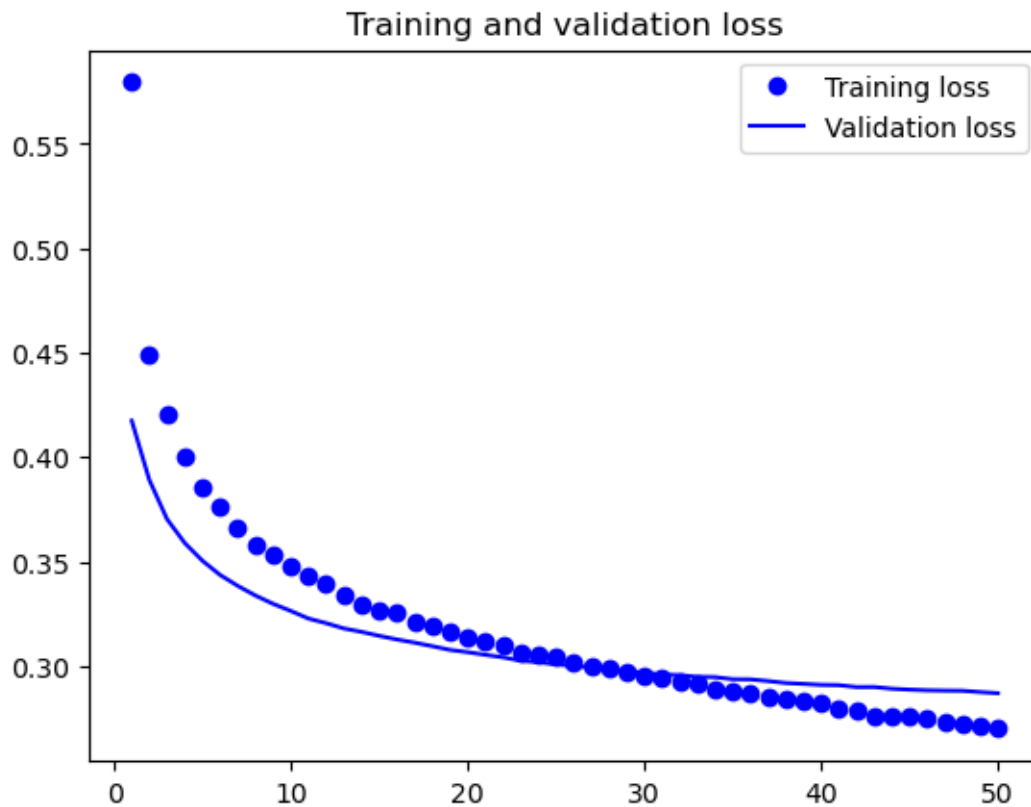
binary_accuracy: 0.8832 - loss: 0.2841 - val_binary_accuracy: 0.8776 - val_loss:
0.2919
Epoch 39/50
251/251          3s 6ms/step -
binary_accuracy: 0.8826 - loss: 0.2831 - val_binary_accuracy: 0.8773 - val_loss:
0.2915
Epoch 40/50
251/251          2s 7ms/step -
binary_accuracy: 0.8841 - loss: 0.2824 - val_binary_accuracy: 0.8771 - val_loss:
0.2910
Epoch 41/50
251/251          2s 7ms/step -
binary_accuracy: 0.8844 - loss: 0.2797 - val_binary_accuracy: 0.8784 - val_loss:
0.2909
Epoch 42/50
251/251          2s 7ms/step -
binary_accuracy: 0.8847 - loss: 0.2787 - val_binary_accuracy: 0.8784 - val_loss:
0.2899
Epoch 43/50
251/251          2s 6ms/step -
binary_accuracy: 0.8863 - loss: 0.2762 - val_binary_accuracy: 0.8781 - val_loss:
0.2900
Epoch 44/50
251/251          3s 6ms/step -
binary_accuracy: 0.8860 - loss: 0.2755 - val_binary_accuracy: 0.8789 - val_loss:
0.2891
Epoch 45/50
251/251          3s 7ms/step -
binary_accuracy: 0.8860 - loss: 0.2759 - val_binary_accuracy: 0.8791 - val_loss:
0.2887
Epoch 46/50
251/251          3s 7ms/step -
binary_accuracy: 0.8867 - loss: 0.2746 - val_binary_accuracy: 0.8792 - val_loss:
0.2884
Epoch 47/50
251/251          2s 6ms/step -
binary_accuracy: 0.8865 - loss: 0.2734 - val_binary_accuracy: 0.8783 - val_loss:
0.2882
Epoch 48/50
251/251          3s 7ms/step -
binary_accuracy: 0.8884 - loss: 0.2723 - val_binary_accuracy: 0.8791 - val_loss:
0.2882
Epoch 49/50
251/251          3s 7ms/step -
binary_accuracy: 0.8873 - loss: 0.2713 - val_binary_accuracy: 0.8790 - val_loss:
0.2875
Epoch 50/50
251/251          3s 7ms/step -

```

binary_accuracy: 0.8883 - loss: 0.2699 - val_binary_accuracy: 0.8790 - val_loss: 0.2870

```
[115]: accuracy = history.history["binary_accuracy"]
val_accuracy = history.history["val_binary_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(accuracy) + 1)
plt.plot(epochs, accuracy, "bo", label="Training accuracy")
plt.plot(epochs, val_accuracy, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





```
[116]: test_model = keras.models.load_model(
        "fine_tuning_resnet50v2.keras")
test_loss, test_acc = test_model.evaluate(
    test_features,
    test_labels,
    batch_size=32
)

print(f"Test accuracy: {test_acc:.3f}")
```

```
33/33          0s 3ms/step -
binary_accuracy: 0.8892 - loss: 0.2723
Test accuracy: 0.889
```

```
[ ]:
```

```
[ ]:
```

5 Performance Analysis

```
[124]: data = {
    'Model Name': [
        'convnet_from_scratch',
        'convnet_from_scratch_with_augmentation',
        'feature_extraction_resnet50v2',
        'fine_tuning_resnet50v2'
    ],
    'Validation Accuracy': [0.8627, 0.8876, 0.8851, 0.8791],
    'Test Accuracy': [0.872, 0.868, 0.890, 0.889]
}

df = pd.DataFrame(data)

# Sorting by Test Accuracy in descending order as per guidelines
df = df.sort_values(by='Test Accuracy', ascending=False)

models = df['Model Name'].tolist()
val_acc = df['Validation Accuracy'].tolist()
test_acc = df['Test Accuracy'].tolist()

x = np.arange(len(models))
width = 0.35

fig, ax = plt.subplots(figsize=(12, 6))
rects1 = ax.bar(x - width/2, val_acc, width, label='Validation Accuracy',
    color='skyblue')
rects2 = ax.bar(x + width/2, test_acc, width, label='Test Accuracy',
    color='navy')

ax.set_ylabel('Accuracy')
ax.set_title('Model Performance Analysis')
ax.set_xticks(x)
ax.set_xticklabels(models, rotation=15, ha='right')
ax.set_ylim(0.8, 0.92) # Zooming in to see the differences better
ax.legend()

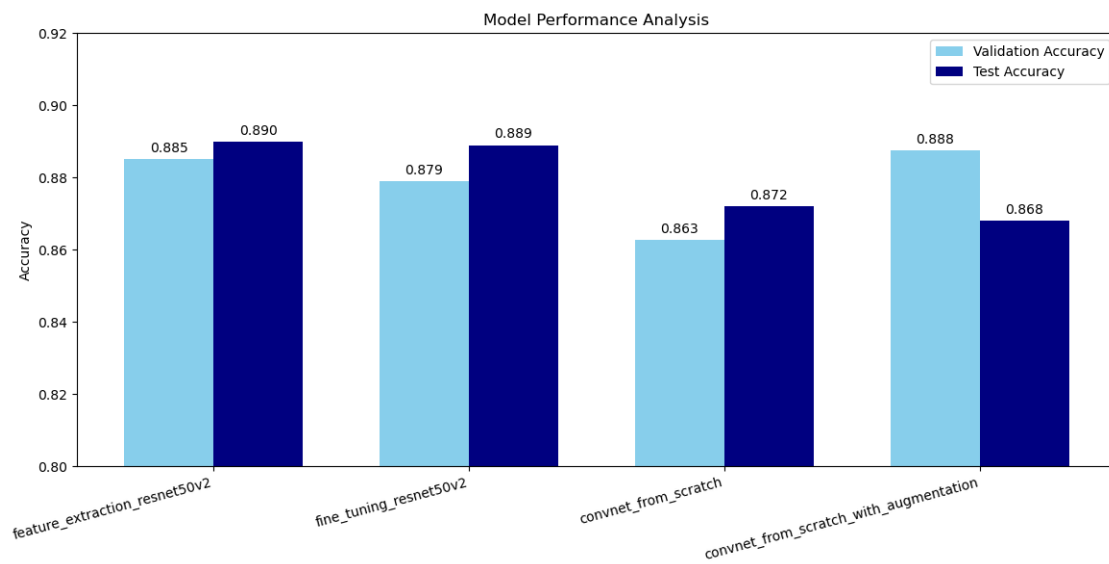
def autolabel(rects):
    for rect in rects:
        height = rect.get_height()
        ax.annotate(f'{height:.3f}',
            xy=(rect.get_x() + rect.get_width() / 2, height),
            xytext=(0, 3),
            textcoords="offset points",
            ha='center', va='bottom')
```



```
autolabel(rects1)
autolabel(rects2)

plt.tight_layout()
plt.savefig('model_performance_chart.png')
print("Chart saved as model_performance_chart.png")
```

Chart saved as model_performance_chart.png



[]: